Energy-Efficient Scheduling for Cattle Collars An Optimized Approach Based on Analytic Hierarchy Process and Preemption Threshold

Pengpeng Sun
Northwest A&F University
Xianyang, Shaanxi, China
jiea489756@nwafu.edu.cn

Hanchen Wang
Northwest A&F University
Xianyang, Shaanxi, China
1275994890@qq.com

Yelan Xing
Northwest A&F University
Xianyang, Shaanxi, China
xylcode@163.com

Bingze Chen
Northwest A&F University
Xianyang, Shaanxi, China
2420519069@qq.com

Xinrong Kou
Northwest A&F University
Xianyang, Shaanxi, China
2519253039@qq.com

Hongming Zhang*
Northwest A&F University
Xianyang, Shaanxi, China
zhm@nwsuaf.edu.cn

Pujing Zhang
Kingbull Livestock Co., Ltd.
Xianyang, Shaanxi, China
office@qinbao.com.cn

Abstract

Embedded microcontrollers with low power consumption, high performance, and rich interfaces widely use FCFS scheduling, which suffers from long average wait times and inefficiency for short tasks, increasing energy consumption. To address the lack of clear priority rules in existing ERPT scheduling, this paper proposes a Hierarchical Priority Preemptive Threshold (HPPT) algorithm based on Analytic Hierarchy Process (AHP). HPPT establishes task priorities through energy metrics, builds a low-energy model for prioritized scheduling, and optimizes system-level energy efficiency. Physical experiments show HPPT reduces device power consumption and extends operational lifespan by approximately 24.08%.

CCS Concepts

• **Computer systems organization** → *Embedded software.*

Keywords

Analytic Hierarchy Process, priority assignment, preemption threshold allocation, embedded device energy consumption, worst-case response time

ACM Reference Format:
Pengpeng Sun, Hanchen Wang, Yelan Xing, Bingze Chen, Xinrong Kou, Hongming Zhang, and Pujing Zhang. 2025. Energy-Efficient Scheduling for Cattle Collars An Optimized Approach Based on Analytic Hierarchy Process and Preemption Threshold. In *Great Lakes Symposium on VLSI 2025 (GLSVLSI '25), June 30–July 02, 2025, New Orleans, LA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3716368.3735249

GLSVLSI '25, New Orleans, LA, USA

ACM ISBN 979-8-4007-1496-2/25/06
https://doi.org/10.1145/3716368.3735249

1 Introduction

With productivity advancing, traditional pastoralism fails to meet modern needs, and smart pastoralism empowered by new-type productivity emerges as a transformative paradigm. In IoT systems, embedded computing modules for livestock data collection and transmission have energy consumption dependent on sensors' baseline power and task scheduling overhead. Reducing energy consumption and extending operational longevity are critical as the number of modules grows. Microcontroller-based IoT devices that only use FCFS scheduling lack autonomous task management, frequently running energy-intensive tasks at short intervals. This raises power demands and increases maintenance costs through frequent battery replacements.

Dynamic Voltage and Frequency Scaling (DVFS) [1, 2] and Dynamic Power Management (DPM) [3–5] can effectively reduce system energy consumption under temporal constraints. However, most embedded devices require continuous long-term operation. Insufficient power risks circuit failures, device damage, and erroneous data uploads contaminating cloud datasets. Battery management challenges include deep discharge shortening lifespan and peak-load power interruptions damaging components irreversibly. Manual battery replacement for large-scale deployments is cost-prohibitive.

Allavena and Mossé [6] pioneered the study of task scheduling in energy-harvesting embedded systems. Subsequent research introduced algorithms such as the Lazy Scheduling Algorithms (LSA) [7] which adjusts CPU frequency based on task energy demands to regulate the Worst-Case Execution Time (WCET). However, LSA's efficacy relies on oversimplified assumptions that task energy consumption correlates directly with WCET, neglecting multifaceted factors like processor frequency, voltage levels, task interdependencies, and system loads [8], limiting its applicability in complex scenarios.

In scheduling theory, As Late As Possible (ALAP) and As Soon As Possible (ASAP) are opposing strategies [9]. ASAP prioritizes early

task completion for resource utilization, while ALAP delays execution to the latest permissible time, both widely used in optimization [10]. Abdeddaïm [11, 12] showed ASAP outperforms ALAP in balancing energy and timing, providing better scheduling solutions. Dong [13], Zhang [14], and Jiang [15] explored low-power circuit design and power network modeling. However, these studies suffer from two critical limitations:

1) They overlook the technical complexity of optimizing power consumption across diverse microcontroller architectures from a user perspective.

2) They inadequately address the heterogeneous characteristics and data timeliness within task sets, and yield suboptimal scheduling solutions. Thus, a more sophisticated scheduling algorithm is imperative to achieve a comprehensive energy management.

Scholars stress low-power optimization for task scheduling and energy management. The Preemptive Threshold Scheduling Algorithm (PTSA) assigns preemption thresholds to tasks, with higher-priority tasks preempting to ensure resource efficiency and real-time response. PTSA combines preemptive/non-preemptive advantages, enhancing schedulability with minimal memory overhead [16] and optimizing stack size [17], while an adaptive energy-aware variant improves threshold allocation in EHCPS [18]. The Energy-Related Preemption Threshold Scheduling (ERPT) uses Worst-Case Response Time (WCRT) analysis for energy efficiency but lacks explicit priority assignment guidelines, hindering embedded applications [19]. Existing research assumes predefined priorities, neglecting priority determination. This paper addresses this by focusing on pre-scheduling priority allocation for PTSA to provide a task priority framework.

Task priority allocation requires multi-factor evaluation to determine weights. An AHP-based method for resource allocation in coupled tasks resolves weight inconsistencies via hierarchical modeling and task coupling analysis [20], informing this paper's priority framework.

The primary contributions are:

1)The Hierarchical Priority Preemptive Threshold (HPPT) algorithm integrating AHP and ERPT unifies priority scheduling, reducing embedded device power consumption by 24.08% via holistic task characterization while ensuring schedulability and timeliness.

2) Simulations and physical experiments validate HPPT's energy-efficiency superiority over FCFS, RR, and ERPT. Power-time correlations are analyzed to support its embedded systems applications.

2 Hierarchical Task Scheduling Problem

The hierarchical task scheduling model is formally defined as follows:

1)Task sets $\tau = \{\tau_1, \tau_2, ..., \tau_m\}$, where $\tau_i = \{C_i, D_i, \pi_i, \gamma_i\}$. Where C_i represents the Worst-Case Execution Time (WCET), D_i represents the relative deadline, π_i represents the task priority, and γ_i represents the preemption threshold.

2)Priority Set $\pi = \{\pi_1, \pi_2, ..., \pi_m\}$, where $\pi_i (i \in (1, m))$ represents the priority of the i-th task. $\pi_i = \{E_i, T_i, S_i, M_i, L_i\}$, where E_i denotes the average energy consumption level, T_i denotes the average time consumption level, S_i denotes the program storage space, M_i denotes the allocated dynamic memory, and L_i denotes the local used variables. Lower numerical priority value indicates higher precedence.

3)Preemption Threshold Set $\gamma = \{\gamma_1, \gamma_2, ..., \gamma_m\}$, where $\gamma_i (i \in (1, m))$ denotes the preemption threshold of the i-th task, dynamically computed prior to each scheduling instance, and there is a constraint $\gamma_i \in [1, \pi_i]$. Tasks with lower preemption threshold values are assigned higher preemption priority. The definitions of the two functions $AHP(E, S, T, M, L)$, $ERPT(\tau, C, \gamma)$ are established based on the Analytic Hierarchy Process (AHP) for the task priority assignment and the preemptive threshold allocation method. Specifically, the $AHP(E, S, T, M, L)$ function takes the average energy consumption level E, program storage space S, average time consumption level T, allocated dynamic memory M, and local used variables L by the task as input parameters. It then computes and returns the numerical priority value according to the specified priority assignment method. The function $ERPT(\tau, C, \gamma)$ contains input parameters: the task structure τ, worst-case execution time C, and the current preemption threshold value γ. This function recalculates the preemption threshold using the allocation method $ERPT(\tau, C, \gamma)$ and returns the updated preemption threshold value for the task. Set a flag variable ism which can take on a value of either 0 or 1 to indicate the schedulability of a task set (1: schedulable, 0: unschedulable). The hierarchical task scheduling problem is formulated as follows. The objective function is defined in formula (1) and its constraints are defined in formulas (2)-(5).

$$(\sum E)_{min} = \sum_{i=1}^{m} \{\pi_i \{E_i\}\} \tag{1}$$

$$\pi_k = AHP(E_k, S_k, T_k, M_k, L_k), k \in [1, m] \tag{2}$$

$$\gamma_k = ERPT(\tau_k, C_k, \gamma_k), k \in [1, m] \tag{3}$$

$$\sum E = \sum_{k=1}^{m} \gamma_k \times E_{\tau_k} \times ism \tag{4}$$

$$\begin{cases} ism = 0, \exists n \in [1, m] : \tau_k > \tau_n \\ ism = 1, \forall n \in [1, m] : \tau_n > \tau_k \end{cases} \tag{5}$$

3 Scheduling Methodology for Hierarchical Task Problem

This paper proposes a Hierarchical Priority Preemptive Threshold (HPPT) strategy to address the task scheduling problem in embedded devices. The workflow is structured as follows:

Initialization phase: defining the task set and execution time limits during initialization; constructing a task attribute hierarchy and judgment matrix; calculating weight vectors via consistency checks to determine task priorities; assigning preemption thresholds using WCRT analysis; and sorting tasks by ascending preemption thresholds to establish execution order.

Execution phase: The system continuously monitors task states and evaluates preemption thresholds. The head-of-queue task with the lowest preemption threshold (highest priority) is dispatched if it meets execution criteria; otherwise, it is suspended and the queue reordered by ascending thresholds. For executing tasks, execution duration is checked against an upper bound. If the limit is not reached, preemption thresholds are dynamically adjusted and the process repeats. Upon reaching the limit, the task terminates, releasing its resources.

An algorithm based on the hierarchical analysis and Preemption Threshold Policy (HPPT) is shown in Algorithm 1.

Algorithm 1 Hierarchical Priority Preemptive Threshold Scheduling Algorithm

Initialize tasks with execution times using InitializeTasks(tasks, execution_times)
PriorityAssignment(tasks(n), alternatives(n))
for $i = 0$ to $n - 1$ **do**
  $tasks[i].preemption\_threshold \leftarrow$ CalculateWorstCaseResponseTime($tasks[i]$)
end for
Sort tasks by preemption threshold using SortTasksByPreemptionThreshold(tasks)
while not all tasks completed **do**
  $head\_task \leftarrow$ GetHeadTask($tasks$)
  if $head\_task$ meets minimum preemption threshold **then**
    ExecuteTask($head\_task$)
  else
    SuspendTask($head\_task$)
    AdjustTaskOrder(tasks)
  end if
  if $head\_task.execution\_time < upper\_limit$ **then**
    AdjustPreemptionThresholds(tasks)
  else
    TerminateExecution($head\_task$)
    ReleaseResources($head\_task$)
  end if
end while
return $tasks[n].priority$, $tasks[n].preemption\_threshold$, $isValid$

3.1 Priority Assignment for HPPT

To conduct pairwise comparisons of the importance of each factor within the task priority quintuple $\{E_k, S_k, T_k, M_k, L_k\}$, the values of the corresponding judgment matrix A are assigned and established. Subsequently, apply the square root method to calculate the weights $\bar{w}$ as shown in Equation (6), where the element a_{ij} is located at the i-th row and j-th column.

$$\bar{w} = \sqrt[n]{\prod_{j=1}^{m} a_{ij}} \tag{6}$$

Normalize the calculated weights as shown in Equation (7).

$$w_i = \frac{\bar{w}}{\sum_{j=1}^{m} \bar{w}_j} \tag{7}$$

Perform a consistency check on the aforementioned vector by calculating the maximum eigenvalue λ_{max} according to Equation (8), and compute the consistency index C.I. following Equation (9).

$$\lambda_{max} = \frac{\sum_{j=1}^{m} \frac{(Aw)_i}{w_i}}{m} \tag{8}$$

$$C.I. = \frac{\lambda_{max} - m}{m - 1} \tag{9}$$

Ultimately, the corresponding value of the average random consistency index R.I. is determined by consulting a table, and the Consistency Ratio (C.R.) is calculated as shown in Equation (10).

$$C.R. = \frac{C.I.}{R.I.} \tag{10}$$

If the Consistency Ratio (C.R.) is less than 0.1, then the consistency check is passed, equation (7) represents the weight vector for task priority indices.

Based on the above analysis, this paper computes the algebraic sum of the products of each factor and its corresponding weight for a set of tasks τ_{nsort}. This computation yields a comprehensive factor value for the priority of each task. These values are then sorted in ascending order to obtain an ordered sequence of tasks ranked by their comprehensive indicator values. The task at the top of this sequence is assigned a priority level of 1, the second task is assigned a priority level of 2, and so on. Ultimately, a task set $\tau = \{\tau_1, \tau_2, ..., \tau_m\}$ with fully allocated priority levels is resulted.

3.2 Preemption Threshold Assignment for HPPT

This paper further analyzes the schedulability of task sets using a worst-case response time analysis method [21, 22]. In this section, the worst-case response time R_i for task τ_i is defined as the maximum value between the worst-case processor demand time W_i^P and the worst-case energy replenishment time $\left\lfloor \frac{W_i^E}{e_{bat}} \right\rfloor$. The latter term refers to the time required to recharge the storage device up to its maximum energy capacity e_{bat}. For simplicity, this paper assumes that the worst-case response time for a task is equal to its fastest processor demand time, i.e., $R_i = W_i^P$.

The process for allocating preemptive thresholds under the HPPT strategy proceeds as follows. Initially, it is assumed that the current task is schedulable. Within a predefined number of iterations, the preemptive threshold of the subsequent task is adjusted multiple times. During each adjustment, the maximum blocking time B_i^P, the start time $S_i^P(q)$, and the end time F_i^P of q-th instance of a task are calculated. Based on these values, the worst-case response time is computed. This WCRT is then compared with the task's deadline. If the WCRT is less than its deadline, the adjustment process of the preemptive threshold is saved, and the task set is resorted in ascending order based on the preemptive threshold values. Conversely, if the WCRT exceeds the deadline, the current adjustment is discarded, and the next iteration begins.

The formula for calculating the maximum blocking time B_i^P is shown in Equation (11).

$$B_i^P = \max_j \{C_i :: \gamma_i \le \pi_i < \pi_j\} \tag{11}$$

The start time $S_i^P(q)$ of the q-th instance of a task τ_i is calculated as shown in Equation (12).

$$S_i^P(q) = B_i^P(\tau_i) + (q-1)(C_i + C_{vcsw}) + \sum_{\substack{\forall j \\ \pi_j < \pi_i}} \left(1 + \left\lfloor S_i^P(q) \right\rfloor\right)(C_j + 2C_{nvcsw}) \tag{12}$$

Where C_{vcsw} represents the voluntary context switch time, and C_{nvcsw} represents the involuntary context switch time.

The end time F_i^P of the q-th instance of a task τ_i is calculated as shown in Equation (13).

$$F_i^P = S_i^P(q) + (C_i + C_{vcsw}) + \sum_{\substack{\forall j \\ \pi_j < \pi_i}} \left(\left\lceil F_i^P(q) \right\rceil - \left(1 + \left\lfloor S_i^P(q) \right\rfloor \right) \right) \times (C_j + 2C_{nvcsw}) \quad (13)$$

The worst-case processor demand time W_i^P for a task τ_i is calculated as shown in Equation (14).

$$W_i^P = \left(F_i^P(q) - q + 1 \right)_{q \in 1,2,\ldots,m} \quad (14)$$

Where the variable m represents the last instance of the task τ_i.

The worst-case response time R_i for a task τ_i is calculated as shown in Equation (15).

$$R_i = W_i^P \quad (15)$$

When the worst-case response time R_i of a task τ_i is less than its deadline D_i, it indicates that the task τ_i still maintains real-time characteristics which means that the task τ_i is schedulable. If all tasks in the task set are schedulable, then the entire task set is considered schedulable.

4 Experiment

4.1 Setting

The experiment employs the ATmega2560-16AU microcontroller as the primary controller. The MLX90614 collects target temperature data, the MAX30102 captures blood oxygen saturation and heart rate, the MPU6050 provides six-axis acceleration data, the ATGM336H delivers GPS coordinates, and the BC260Y uploads all collected device data. The task set are presented in Table 1.

Table 1: Parameters related to each task

Chips	E_i/mAh	T_i/ms	S_i/Byte	D_i/Byte	L_i/Byte
MLX90614	0.48	0.2702	4.46	0.39	7.61
MAX30102	0.30	0.2499	10.6	2.20	5.79
MPU6050	0.24	0.2793	9.69	0.50	7.49
ATGM336H	7.20	1.0930	9.29	0.42	7.58
BC260Y	42.0	12.010	11.0	0.86	7.13

This section demonstrates the HPPT strategy's power-saving efficacy by modeling embedded device tasks as structures and allocating them to queues via scheduling algorithms, with completed tasks looping back for repeated execution.

Simulations use C++ to model task execution under various scheduling algorithms, while physical experiments employ EDA software to design PCBs integrating all chips, with corresponding scheduling algorithm code programmed for execution. The following is a brief description of the scheduling algorithm used:

1)First Come First Serviced (FCFS) & Round Robin (RR): For the FCFS scheduling algorithm, tasks are arranged in ascending order based on their task set numbering to form the task queue. For the RR scheduling algorithm, an additional time slice of 0.275 ms is introduced on top of the ascending order task queue. If any task in the task set requires more time than the time slice size, the current progress of the task is saved, and the process resources are preempted to the next task.

2)Fair Sharing (FS) Resource Scheduling Algorithm: Set the reference time slice size to 2.4 ms. Add two integer variables Ncount and Ycount, into the task structure, and both variables are initialized to 1. Ncount records the number of times which the task has not been completed within its time slice, while Ycount records the number of times which the task has been completed within its time slice. During task execution, the minimum value between the reference time slice and the remaining execution time ($\frac{Ncount}{Ycount+Ncount}$) is taken as the allocated time slice for this task. The rest of the execution process remains the same as RR.

3)Energy Related Preemption Threshold (ERPT) Scheduling: By default, each task in the current task set has a priority of 1. After a task is completed, its priority is not adjusted, and the task set continues to execute in a loop.

4)Hierarchical Priority Preemptive Threshold (HPPT) Scheduling Algorithm: Assign specific priorities and preemption thresholds to each task within the task set. During task execution, check if the preemption threshold of the current task is greater than the priority of the next task. If this condition is met, perform a preemption scheduling by swapping the order of the two tasks and then execute the current task. After completion, dynamically adjust the preemption threshold based on the state of the current task set.

For the allocation of task priorities, it is first necessary to determine the values of the judgment matrix. Based on a quantitative assessment of the relative importance of each level of factors, a scale system from 1 to 9 is adopted with the scale value being positively correlated with the weight. A value of 1 indicates that two indicators have equal weight. By evaluating each pair of indicators in the quintuple $\{E_k, S_k, T_k, M_k, L_k\}$, an example of constructing a judgment matrix is shown in Equation (16).

$$A = \begin{bmatrix} 1 & 1/2 & 1/5 & 1/4 & 1/6 \\ 2 & 1 & 1/4 & 1/3 & 1/4 \\ 5 & 4 & 1 & 2 & 1 \\ 4 & 3 & 1/2 & 1 & 1/2 \\ 6 & 4 & 1 & 2 & 1 \end{bmatrix} \quad (16)$$

After performing weight calculations and normalization, maximum eigenvalue calculations, consistency index calculations, and consulting the average random consistency index, C.R.=0.0102<0.1 is obtained to indicate that the consistency check has been passed. Thus, the weight vector corresponding to matrix A is adopted as the task priority indicator weight vector.

The simulation experiment uses random seeds to generate 50 tasks (30 low-complexity tasks and 20 high-complexity tasks), and the physical experiment uses 5 tasks from the data in Table 1. Due to space limitations, the simulated task table starts from the 10th task and displays entries at intervals of 10 tasks, while the physical experiment table is presented in full. Under the priority assignment algorithm in Section 3, the priorities of the tasks are shown in Tables 2 and 3.

This paper uses the task preemption threshold allocation algorithm [23] to allocate preemption thresholds to the task sets after

Table 2: Simulated task set with priority hierarchy analysis

TP	E_i/mAh	T_i/ms	S_i/Byte	D_i/Byte	L_i/Byte	EV	V
π_{10}	0.28	0.519	5.12	0.44	5.58	5.26	8
π_{20}	0.16	0.427	3.38	0.70	5.29	6.71	13
π_{30}	1.43	4.060	12.6	4.33	49.6	16.8	19
π_{40}	0.17	0.829	4.46	0.46	6.69	5.31	9
π_{50}	3.15	5.290	9.98	5.59	65.2	26.4	42

Table 3: Physical task set with priority hierarchy analysis

TP	E_i/mAh	T_i/ms	S_i/Byte	D_i/Byte	L_i/Byte	EV	V
π_1	0.48	0.270	4.46	0.39	7.61	4.18	1
π_2	0.30	0.250	10.6	2.20	5.79	6.38	2
π_3	1.24	0.279	9.69	0.50	7.49	6.89	3
π_4	7.20	1.093	9.29	0.42	7.58	7.04	4
π_5	42.0	12.01	11.0	0.86	7.13	11.2	5

their priority hierarchy has been analyzed. The results are presented in Table 4 and 5.

Table 4: Simulated preemption threshold allocation

Tasks	C_i/ms	W_i/ms	π_i	R_i/ms	γ_i
τ_{10}	0.519	0.52	8	5.1471	2
τ_{20}	0.427	0.43	13	10.109	3
τ_{30}	4.060	0.41	19	14.761	15
τ_{40}	1.093	1.10	9	19.284	5
τ_{50}	5.290	5.30	42	23.571	17

Table 5: Physical preemption threshold allocation

Tasks	C_i/ms	W_i/ms	π_i	R_i/ms	γ_i
τ_1	0.270	0.30	1	0.004	1
τ_2	0.250	0.30	2	0.274	1
τ_3	0.289	0.30	3	0.528	2
τ_4	1.093	1.15	4	0.811	4
τ_5	12.01	12.5	5	1.908	5

4.2 Simulation

The total runtime is set to a duration of 10^7 milliseconds (ms). After one complete cycle of the task set execution, the device rests for 10 ms. The energy consumption within this duration is used as a metric to evaluate the effectiveness of different strategies. Figure 1 shows the energy consumption of each strategy within the specified duration 140 ms.

Integrating the energy consumption over the specified duration 10^7 ms yields the following results: FCFS consumes 8.46244 mAh, ERPT consumes 8.09747 mAh, RR consumes 7.91786 mAh, FS consumes 7.73416 mAh, and HPPT consumes 7.46027 mAh. The

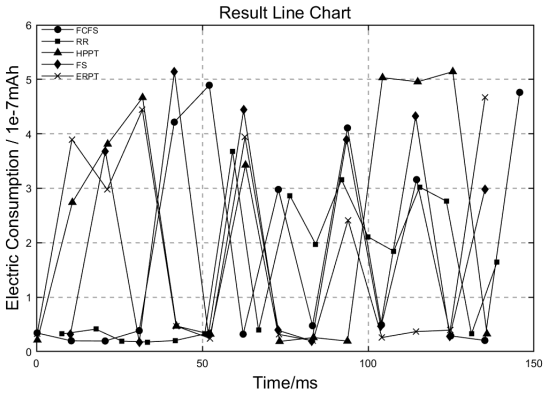

Figure 1: Energy Consumption of Different Scheduling Strategies

result of different scheduling strategies for energy consumptions is illustrated in Figure 2.

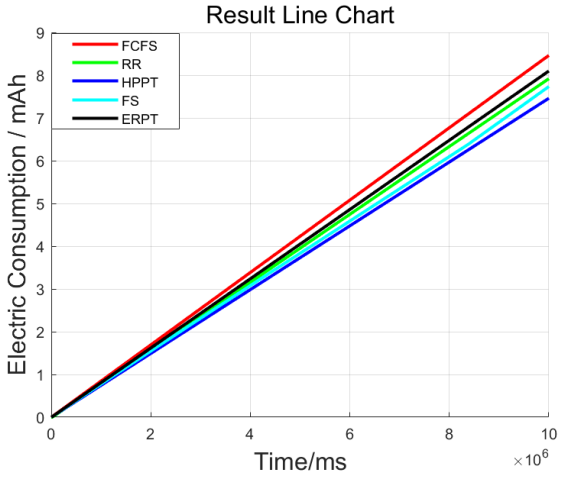

Figure 2: Integrated Energy Consumption of Different Scheduling Strategies

As shown in Figure 2, the HPPT strategy significantly reduces the slope of the energy consumption-time function compared to other scheduling strategies, indicating lower power consumption per unit time. Within 2.7 hours of operation, HPPT achieves a 1.00217 mAh reduction in energy consumption versus the default FCFS strategy on an AVR device, representing an 11.84% improvement. These results confirm HPPT's effectiveness in optimizing energy efficiency and provide critical evidence for enhancing energy utilization in embedded systems.

4.3 Physical Experiment

To determine the energy-saving effects of the HPPT strategy in embedded devices, this paper utilizes an EDA software to design and print integrated modules. Tasks(Table 3) execution codes for different scheduling strategies, as used in the physical experiments,

are written using the Arduino IDE and then burned onto the printed circuit boards (PCBs). The system was powered by a 3500 mAh 18650 lithium-ion battery until the battery voltage dropped below 3.0V. The runtime under different strategies is compared and the results are shown in Table 6.

Table 6: Comparison of Operation Results

S	EC/mA	DT/h
FCFS	8.020	436.41
ERPT	7.660	456.92
RR	7.413	472.14
FS	7.192	486.65
HPPT	6.463	541.51

Where S denotes scheduling strategies, EC represents energy consumption per hour, and DT indicates duration time of each scheduling strategy when the device is powered by the lithium-ion battery. Results show that the FCFS strategy achieves 436.41 hours of execution on an Arduino Mega2560 with a 3500 mAh battery, while HPPT extends this to 541.51 hours under identical conditions. This experiment demonstrates that HPPT increases embedded device operational time by 24.08% compared to FCFS for equivalent battery capacity.

5 Conclusion

From the comparison of the simulation result and the physical experimental result, the HPPT strategy does indeed have certain energy-saving effects compared to the FCFS strategy, but the significance of its effectiveness may depend on the characteristics of the tasks themselves and the scale of the task set. In this case, the energy-saving effect of the HPPT strategy may be slightly inferior to that of strategies such as FS, RR, or other currently more advanced scheduling algorithms on certain task sets, and this result still requires validation.

This paper addresses a critical limitation of the Energy-Related Preemption Threshold (ERPT) strategy—the absence of explicit task priority allocation—rendering conventional energy-aware scheduling ineffective for embedded systems. To resolve this, the Hierarchical Priority Preemptive Threshold (HPPT) strategy is proposed, integrating the Analytic Hierarchy Process (AHP) with rigorous consistency validation to systematically assign task priorities. Simulations and experiments demonstrate that HPPT significantly reduces energy consumption and extends operational longevity in multi-module embedded systems, establishing its potential as a robust scheduling framework for energy-constrained environments.

However, although validated on specific embedded platforms, the scalability of HPPT to diverse architectures (e.g., high-performance processors) and large-scale task sets remains unexplored. Future work should explore HPPT's adaptability to heterogeneous systems and its effectiveness in managing complex, real-time scheduling scenarios.

References

[1] J. Zidar, T. Matić, I. Aleksi, and Ž. Hocenski. 2024. Dynamic voltage and frequency scaling as a method for reducing energy consumption in ultra-low-power embedded systems. *Electronics* 13, 5 (Mar 2024), 826.

[2] Z. Zhang, Y. Zhao, H. Li, C. Lin, and J. Liu. 2024. DVFO: Learning-Based DVFS for Energy-Efficient Edge-Cloud Collaborative Inference. *IEEE Transactions on Mobile Computing* 23, 10 (Oct 2024), 9042–9059.

[3] S. Khriji, R. Chéour, and O. Kanoun. 2022. Dynamic voltage and frequency scaling and duty-cycling for ultra low-power wireless sensor nodes. *Electronics* 11, 24 (Dec 2022), 4071.

[4] H. Khan, I. Din, A. Ali, and M. Husain. 2023. An optimal DPM based energy-aware task scheduling for performance enhancement in embedded MPSoC. *Computers, Materials & Continua* 74, 1 (Jan 2023), 2097–2113.

[5] H. Ali and K. Javad. 2023. A multifunctional complex droop control scheme for dynamic power management in hybrid DC microgrids. *International Journal of Electrical Power & Energy Systems* 152 (Aug 2023), 109224.

[6] M. Benzaouia et al. 2023. Real-time super twisting algorithm based fuzzy logic dynamic power management strategy for hybrid power generation system. *Journal of Energy Storage* 63 (Jun 2023), 107316.

[7] A. Allavena and D. Mosse. 2001. Scheduling of frame-based embedded systems with rechargeable batteries. In *Proceedings of the Workshop on Power Management for Real-Time Embedded Systems*. IEEE, Washington, DC, USA, 1–8.

[8] Y. Dong et al. 2020. Lazy scheduling-based disk energy optimization method. *Tsinghua Science and Technology* 25, 2 (Apr 2020), 203–216.

[9] R. Jayaseelan, T. Mitra, and X. Li. 2006. Estimating the worst-case energy consumption of embedded software. In *Proceedings of the 12th IEEE Real-Time and Embedded Technology and Applications Symposium*. IEEE, Washington, DC, USA, 81–90.

[10] K. Baital and A. Chakrabarti. 2024. Energy-efficient dynamic scheduling of dependent tasks for multi-core real-time systems using delay techniques. *Concurrency and Computation: Practice and Experience* 36, 27 (May 2024), e8267.

[11] H. Nishikawa, K. Shimada, I. Taniguchi, et al. 2022. Mouldable fork-join task scheduling techniques with inter and intra-task communications. *International Journal of Embedded Systems* 15, 1 (Mar 2022), 69–81.

[12] Y. Abdeddaïm, Y. Chandarli, and D. Masson. 2013. The optimality of PFPasap algorithm for fixed-priority energy-harvesting real-time systems. In *Proceedings of the 25th Euromicro Conference on Real-Time Systems*. IEEE, Piscataway, NJ, USA, 47–56.

[13] Y. Abdeddaïm, Y. Chandarli, R. I. Davis, and D. Masson. 2014. *Approximate response time for fixed priority real-time systems with energy harvesting*. Technical Report hal-00986340. University Grenoble Alpes, Grenoble, France. 1–19 pages.

[14] F. Dong. 2020. Analysis of applications for ultra-low power integrated circuits. *Electronic Components and Information Technology* 4, 2 (Apr 2020), 133–135.

[15] S. Zhang, Y. Du, and W. Yang. 2023. Low-voltage low-power analog integrated circuit design technologies and prospects. *China Integrated Circuit* 32, 8 (Aug 2023), 20–25.

[16] R. Jiang, H. Li, Y. Liu, and J. Liu. 2023. Modeling and analysis of digital integrated circuit low-power supply networks. *Microelectronics & Computer* 40, 7 (Jul 2023), 111–117.

[17] M. Qu. 2021. *Optimization methods for scheduling in mixed-criticality systems based on preemption threshold mechanism*. Ph. D. Dissertation. Nanjing University of Science and Technology, Nanjing, China.

[18] C. Wang. 2016. *Research on memory system optimization algorithms based on preemption threshold scheduling in hard real-time systems*. Ph. D. Dissertation. Zhejiang University, Hangzhou, China.

[19] Y. Ge, Y. Dong, J. Zhang, et al. 2015. An adaptive scheduling algorithm for energy harvesting embedded systems. *Journal of Software* 26, 4 (Apr 2015), 819–834.

[20] Y. Ge, Y. Dong, and B. Gu. 2015. Preemption threshold scheduling for energy harvesting cyber-physical systems. *Journal of Computer Research and Development* 12 (Dec 2015), 2695–2706. Volume number not available.

[21] Q. Tian, C. Huang, H. Yu, et al. 2018. A method for determining resource allocation weights of coupled task sets based on analytic hierarchy process. *Computer Engineering and Applications* 54, 21 (Nov 2018), 25–30, 94.

[22] G. Xu and Y. Liu. 2003. Transaction scheduling of embedded real-time databases based on preemption threshold. *Journal of Huazhong University of Science and Technology (Natural Science Edition)* 31, 6 (Jun 2003), 74–77.

[23] H. Ben and S. Ben. 2020. A parameterized formal model for the analysis of preemption-threshold scheduling in real-time systems. *IEEE Access* 8 (Mar 2020), 58180–58193.